

Assignment 5: Hand Pose Detection with a CNN (20P)

Goals

You roughly know how neural networks and specifically CNNs work. You know how to train a neural network and how to use it for predictions. You know how to change hyperparameters for training neural networks. You know how to measure the quality of a neural network's predictions. You can build a simple application that uses a neural network and apply basics of computer vision.

1 Exploring Hyperparameters

5P

Experiment with hyperparameters to explore their influence on the prediction accuracy of a CNN. As a starting point, use the CNN from the notebook we used in the course and a subset of the HaGRID dataset (see GRIPS). (Don't commit the dataset!)

Now, explore the influence of hyperparameters during training on prediction accuracy for a test dataset, as well as inference time for predictions. Possible hyperparameters are:

- image resolution
- image color mode
- batch size
- number of neurons in the fully connected layers
- number of training images
- number of categories
- activation function of the convolution kernels
- activation function of the fully connected layers
- kernel size for the convolution layers
- number of convolution layers
- learning rate
- data augmentation

You will be assigned one of those hyperparameters. Select at least five different values for your hyperparameter, train a network with each, and log performance metrics. Before training, briefly describe your approach and your assumptions in the notebook. After training, report your findings and visualize your results appropriately. Try to find explanations for your results and discuss them briefly. We will collect your findings in the next session.

Score

- (1P) selected hyperparameter values make sense
- (1P) networks were trained correctly
- (1P) there is code for evaluation and there are results
- (2P) results are reported and visualized appropriately

2 Gathering a Dataset

5P

Extend the HaGRID dataset by capturing at least three distinct images (1920×1080 px) of hand gestures for at least each of the following categories: *like*, *dislike*, *stop*, *rock*, *peace*.

Capture these images in varying environment, light conditons, ... Your images should be similar to those in the dataset.

Then, annotate your dataset and store annotations in a JSON file called *annot-[your_name].json* with a format compatible to the annotations of the original dataset. If there are multiple hands in an image, annotate hands without a gesture as *no_gesture*. You don't have to annotate hand landmarks, ... only label and bounding box are necessary. You can of course annotate those images manually, but you can also write and/or use a program for annotation. Feel free to collaborate with fellow students to implement such a tool – this will not be part of the grading. Commit both the images and annotation file.

Use a CNN trained on (a subset of) the HaGRID dataset to make predictions for your images. Plot the results as a confusion matrix and save it as *conf-matrix.png*.

Score

- (1P) sufficient images captured
- (2P) sufficient images annotated
- (1P) annotations are compatible to the HaGRID dataset
- (1P) confusion matrix

3 Gesture-controlled Camera App

10P

Train a classifier that can distinguish at least three different hand poses. Based on this classifier, create a gesture-controlled camera application in Python: By performing a hand gesture, a selfie timer starts and counts down for a given amount of time. After this time has elapsed, an image is captured and saved to a given path.

Your camera application should support at least two more features. You should be able to toggle them by performing the assigned gesture. Here is some inspiration:

- add nice filter
(e.g., convert to line drawing, sepia, wavy, add fancy border, ...)
- portrait mode
(blur everything but the center or, more advanced, use contour detection for that)
- zoom in/zoom out
- long exposure simulation

Score

- (3P) three hand poses are tracked and distinguished reliably¹
- (1P) selfie timer counts down and saves the result to a given path
- (1P) both time and path can be specified via command line parameters
- (3P) two additional and creative features are implemented
- (1P) mapping of gestures to camera features works and makes sense
- (1P) low latency between gesture and the system's reaction

¹In case finding the user's hand in the camera image is a problem, you can for example use a white wall as a background or use the cardboard with the four markers to act like a physical bounding box.